Concurrent extrema and successor/predecessor queries in search trees

Vladimir Melnikov
EPFL

Gal Sela
Technion, Israel

Abstract

Search trees are a widely used data structure, yet adapting them to a multithreaded environment is non-trivial. While the simplest approach using coarse-grained locking enables the straightforward reuse of single-threaded algorithms, it suffers from significant performance limitations. On the other hand, lock-free search trees offer better scalability but come with high implementation complexity and are difficult to adapt to standard algorithms.

This paper presents a lock-free algorithm for finding extrema (minimum and maximum), as well as predecessor and successor elements, in a wide class of lock-free search trees. We achieve this by implementing an auxiliary structure called the "Ordered Hinted List." This structure maintains an ordered linked list and supports efficient element insertion and removal using "hints."

2012 ACM Subject Classification Computing methodologies → Shared memory algorithms; Computing methodologies → Concurrent algorithms; Theory of computation → Data structures design and analysis

Keywords and phrases Concurrent Algorithms; Concurrent Data Structures; Aggregate queries; Binary Search Tree; Linearizability

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

Algorithms for finding the minimum, maximum, predecessor, and successor in search trees are widely used in various fields. Prior to this work, fast, specialized concurrent solutions for this problem were lacking. While there is extensive research on aggregation functions—specifically, maintaining the element count [7] or using generalized aggregation functions on augmented trees [4]—this approach is often suboptimal for this specific task.

Searching for the maximum and minimum can be significantly optimized because these operations possess unique properties distinct from general aggregation. For instance, when inserting random elements, the global maximum rarely changes. Exploiting these properties allows for high operation locality, avoiding the contention inherent in general aggregation schemes where updates must typically propagate up to the tree root.

The algorithm proposed in this paper adapts a strategy that works well for single-threaded structures to a multithreaded environment. The core idea is to link the tree elements into a linear list. Consequently, searching for the maximum, minimum, successor, and predecessor reduces to operations on this list, which are significantly simpler to implement.

Our approach builds upon Harris's list. Crucially, all synchronization and correctness guarantees rely on a modified version of this list, while the tree structure is used solely to store "hints," whose reliability and strictly consistent state are not mandatory. During the development of this algorithm, we identified that the list component could be isolated as a distinct, efficient lock-free doubly linked list. This list requires only two CAS operations for insertion and three for removal in the contention-free case, outperforming existing algorithms for multithreaded doubly linked lists.


© Vladimir Melnikov and Gal Sela;
licensed under Creative Commons License CC-BY 4.0


Conceptually, the algorithm implements a specialized list that supports local traversal starting from hints provided by an auxiliary structure (typically a tree). Most trees adapt easily to this algorithm; however, challenges arise with some implementations, such as Lock-free relaxed (a,b)-tree [1]. This specific tree implementation sometimes leaves "empty" nodes; reaching such a node fails to provide an approximate value for a key close to the search target. However, standard binary search trees typically return a "close" element even upon a failed search. It is important to note that in this context, "close" does not imply a strict predecessor or successor; it simply requires an element that speeds up traversal by allowing the search to start near the target key rather than from the list head.

2 Core Data Structure Design

To begin, let us describe the structure of the list. In essence, it is a singly linked list where the state of elements is determined by “magic” bits in the pointers to the next element, exactly as in Harris’s [5] list. However, this list also maintains pointers to the previous node. Often, these may be incorrect (in the sense that they do not point to the strictly preceding node in the linearization of the list’s operations). However, they remain in a state close to correct and maintain several key invariants that still allow them to be used for traversing the list backwards without serious issues.

Any node in the list can be in one of two states: Ok and Deleted. In an ideal world, where all operations are atomic and strictly ordered (occurring exactly in the desired order rather than randomly), and where threads do not stall, all nodes would remain in the Ok state for a long time, and upon deletion, instantly pass through the Deleted state. These 2 states work exactly the same as in Harris’s list.

The interesting part of the algorithm is the maintenance of back-pointers. In this implementation, they are susceptible to the ABA problem, so instead of a standard atomic pointer, the algorithm uses a collisionless pointer. In essence, this is the same pointer, for which a CAS will fail if the value, although identical, has changed since the moment of reading. For clarity, when we want to compare specifically the address to which this pointer points, we will use the function unwrap(). It can be implemented using a version counter and Double-Word CAS, or using “indirect pointers” — pointers to a heap-allocated object containing one pointer to the desired location.

Listing 1 presents the key structures used in the algorithm. MarkedPtr - a combination of a pointer and a State that fits into a single CAS word. HintedOrderedList - the structure itself, stores the allocated first and last elements of the list. They cannot be modified or deleted. NodeSnapshot - a structure carrying information about a node and its last known state (pointer to the next and state), widely used in the algorithm for convenience.

Listing 1 Key data structures

```
using CollisionlessPtr = Node*; // explained in the text

enum State {
  Ok,
  Deleted,
};

struct Node {
  atomic<CollisionlessPtr> prev_;
  atomic<MarkedPtr> next_;
  Key key_;
  Value value_;
```

```
13 };
14
15 struct MarkedPtr {
16   Node *ptr_;
17   State state_;
18 };
19
20 struct HintedOrderedList {
21   Node head_;
22   Node tail_;
23 };
24
25 struct NodeSnapshot {
26   Node *self_;
27   MarkedPtr marked_next_;
28 };
29
30 NodeSnapshot take_snapshot(Node *node) {
31   return NodeSnapshot {
32     .self_ = node,
33     .marked_next_ = READ(node->next_),
34   };
35 }
```

Also, the following variation of CAS is used in the algorithm: bool CAS(T &x, T &y, T z). It is equivalent to the atomic execution of the following function:

Listing 2 Semantics of the CAS operation used

```
36 bool CAS(T &x, T &y, T z) {
37   if (x == y) {
38     x = z;
39     y = x;
40     return true;
41   } else {
42     y = x;
43     return false;
44   }
45 }
```

2.1 Contention-free case

For a simple example, let us consider the case of inserting a new node into a list consisting entirely of nodes in the Ok state with completely correct back-pointers.

Insertion begins almost exactly as in Harris's list: to insert a node between previous and the subsequent next, state := READ(previous->next_) (in our example, state == Ok). The new node (new_node) is first initialized with prev_ = previous and next_ = {next, state}, followed by an attempt to insert it via CAS(previous->next_, {next, Ok}, {new_node, Ok}). However, since we must now maintain back-pointers, we must subsequently update the back-pointer of the next node from previous to new_node. This is also done via CAS: CAS(next->prev_, previous, new_node).

Deletion proceeds very similarly to the original Harris list. We transition the node scheduled for deletion (current) into the Deleted (Marked) state using CAS, after which the node is considered deleted. Following this, the link from the previous node (previous) must be changed to point to the next node (next := current->next_.ptr_): CAS(previous->next_, {current, Ok}, {next, Ok}). Afterward, the back-pointer of next must be corrected: CAS(next->prev_, current, previous).

It is important to note that the Deleted state is, in a certain sense, immutable. If a node is in the Deleted state, its state and its pointer to the next element cannot be changed.

2.2 Handling contention: barriers

Obviously, the situation becomes complex only when contention arises. To guarantee the correct behavior of back-pointers, we will use so-called **barriers**. A barrier is a function that interrupts all back-pointers passing *past* a specific node. By virtue of how the algorithm works, once such links are interrupted, new links passing past the node do not appear. 3 presents the code for such a barrier function.

Listing 3 Barrier function

```
46 bool barrier(NodeSnapshot& current, NodeSnapshot& to_check = current) {
47   CollisionlessPtr backpointer = READ(current.marked_next_.ptr_->prev_);
48   NodeSnapshot check_snapshot = take_snapshot(to_check.self_);
49
50   if (to_check != check_snapshot) {
51     to_check = check_snapshot;
52     return false;
53   }
54   if (unwrap(backpointer) == current.self_) return true;
55
56   CAS(current.marked_next_.ptr_->prev_, backpointer, current.self_);
57   return unwrap(backpointer) == current.self_;
58 }
```

To implement these barriers easily, we initially restrict the permissible variations of where back-pointers can point. Figure 1 shows four possible variants: the zero variant is the simplest, and the other three are minor deviations from it that we can easily handle. All other variants are impermissible in the list.

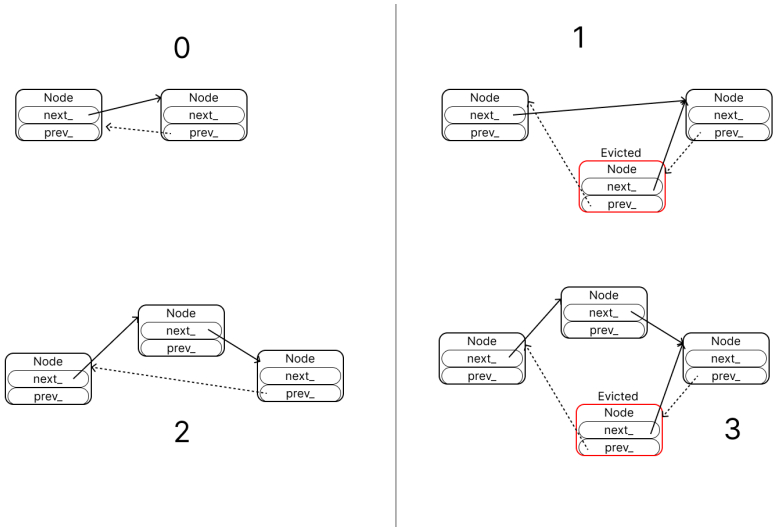


Figure 1 possible states of back-pointers

2.3 Insertion and deletion operations

Let us first consider the insertion function. Before inserting node B between nodes A and C, we must first ensure that the list is in a clean state. We use normalize_next_node to

guarantee that C is not deleted. Then we call the barrier function on A and on C. After this, we insert node B using the standard method via CAS. Subsequently, we call the barrier function on the already inserted node B.

Now let us consider the deletion of node current, which is located between nodes prev and next. First, we ensure the surrounding nodes are in a correct state using normalize_next_node. Then we mark current for deletion using CAS. Once the node is marked deleted, it is logically removed. The physical removal (unlinking) is performed after that by calling helper functions on this marked node.

To help the node complete its deletion (physically remove it), we must call the barrier function on nodes prev, current, and next. And after this, change the pointer prev->next_ to {next, Ok} via CAS.

Listing 4 describes functions for attempting to insert or delete a node.

Listing 4 Functions for inserting and deleting nodes

```
// A (current) is an old node, B (new_node) is a new node
bool try_insert_after(NodeSnapshot &current, Node* new_node) {
  if (current.marked_next_.state_ != Ok) {
    return false;
  }

  NodeSnapshot next = take_snapshot(current.marked_next_.ptr_);

  // Ensure the next node is valid (not deleted)
  if (!normalize_next_node(current, next)) {
    return false;
  }

  // Barriers
  if (!barrier(current)) return false;
  if (!barrier(next)) return false;

  // Initialize new node
  new_node->prev_ = CollisionlessPtr(current.self_);
  new_node->next_ = {next.self_, Ok};

  if (!CAS(current.self_->next_, current.marked_next_, {new_node, Ok})) {
    return false;
  }

  NodeSnapshot inserted = {.self_ = new_node, .marked_next_ = {next.self_, Ok}};
  barrier(inserted);

  return true;
}

bool try_delete(NodeSnapshot &prev, NodeSnapshot &current) {
  assert(prev.marked_next_.ptr_ == current.self_);
  assert(prev.marked_next_.state_ == Ok);

  NodeSnapshot next = take_snapshot(current.marked_next_.ptr_);

  // Normalize before deletion
  if (!normalize_next_node(current, next)) {
    return false;
  }

  // Mark logically deleted
  bool ret = CAS(current.self_->next_, current.marked_next_, {next.self_, Deleted});

```

```
104   if (!ret) return false;
105
106   help_deleted_next(prev, current);
107   return ret;
108 }
109
110 // Ensures that 'current' is followed by a non-deleted node
111 bool normalize_next_node(NodeSnapshot &current, NodeSnapshot &next) {
112   while(next.marked_next_.state_ != Ok) {
113     if(current.marked_next_.state_ != Ok) {
114       return false;
115     }
116
117     // Physically remove the deleted 'next' node
118     help_deleted_next(current, next);
119
120     if (current.marked_next_.ptr_ != next.self_) {
121       next = take_snapshot(current.marked_next_.ptr_);
122     }
123   }
124   return current.marked_next_.state_ == Ok;
125 }
126
127 bool help_deleted_next(NodeSnapshot &current, NodeSnapshot &next) {
128   if (next.marked_next_.state_ == Ok) {
129     return true;
130   }
131   NodeSnapshot nn = take_snapshot(next.marked_next_.ptr_);
132   if (!evict_deleted(current, next, nn)) {
133     return false;
134   }
135   // Fix backpointer of the node after the evicted one
136   post_evict_fix(current, next);
137   return true;
138 }
139
140 bool evict_deleted(NodeSnapshot &prev, NodeSnapshot &current, NodeSnapshot &next) {
141   assert(current.marked_next_.state_ == Deleted);
142   assert(prev.marked_next_.ptr_ == current.self_);
143   assert(current.marked_next_.ptr_ == next.self_);
144
145   // Barriers check for all involved nodes
146   if (!barrier(prev)) return false;
147   if (!barrier(current, prev)) return false;
148   NodeSnapshot &to_check = next.marked_next_.state_ == Ok ? next : prev;
149   if (!barrier(next, to_check)) return false;
150
151   // Physical removal
152   return CAS(prev.self_->next_, prev.marked_next_, {next.self_, Ok});
153 }
154
155 void post_evict_fix(NodeSnapshot &prev, NodeSnapshot &evicted_node) {
156   Node& next_node = *prev.marked_next_.ptr_;
157
158   CollisionlessPtr backptr = READ(next_node.prev_);
159   if (unwrap(backptr) == evicted_node.self_) {
160     CAS(next_node.prev_, backptr, prev.self_);
161   }
162 }
```

2.4 Search functions

Now let us describe the search functions. First, we classify them into two types: read-only and non-read-only. We will begin with non-read-only functions.

2.4.1 Active Traversal Strategy

Starting from the hint, we iterate backwards via prev_ pointers until we find a living node. Starting from it, we iterate forward via next_ pointers until we find the desired key or exceed it. On each iteration, we normalize the next node via the help_normalize_next function. If the current node becomes deleted during this process, we take it as a new hint and restart the whole process. We also call help_normalize_next on the next node, relative to the found one; if an error occurs, we restart. This algorithm is the base for every other non-read-only search function.

To get a successor, we call this algorithm from the current node's key. After it finds the node, we call help_normalize_next on the next one, and if it is successful, we return the next node as the successor (or its value).

To get a predecessor, we call this algorithm with the "slightly less key". It can be implemented as a separate class which, in comparison with another key, behaves like the value it holds, until it is compared with the key equal to its internal value; in that case, it returns that it is less. So calling this algorithm with such a key will return the predecessor.

To implement search, we call this algorithm with the desired key. If the found node's key equals the desired key, we return it as found; otherwise, it is not found.

2.4.2 Read-Only Traversal

Now let us describe the read-only search functions. It is based on finding an "immutable range" which starts with a living node A and ends with another node B, such that there are no living nodes between them. B can be either living or deleted. If we read this range from beginning to end and then check A->next_ to be the same as we read before, we can be sure that this range didn't change in the process of reading it.

So the base of the algorithm is pretty close to the non-read-only one, but instead of normalizing next nodes, we just read nodes consequently and keep track of the last living node we encountered. After we exceed the desired key, we check that the last living node's next_ is the same as we read before. If it is, we return this range as found; otherwise, we restart with this last alive node as a hint.

To implement search, we call this range-finding function with the desired key, and if the found range's first node's key equals the desired key, we return it as found; otherwise, it is not found.

To get a successor, we call this range-finding function with the current node's key but stop not on the first node with a key greater than the desired one, but on the first living node after it. After we get the range, we return the end of this range as the successor.

To get a predecessor, we call this range-finding function with the "slightly less key", as in the non-read-only version, and return the start of the found range as the predecessor.

2.5 Optimizations

There are several optimizations that do not negatively affect performance in high contention scenarios but can significantly accelerate low contention cases.

The first optimization targets the find operation. To determine whether a node is present in the list, it is not necessary to find an "immutable range". Instead, it suffices to find any living node with a key less than or equal to the desired key and then iterate forward until the desired key is found or exceeded. If the current node has the desired key and is alive, we return it as the result. Otherwise, we conclude that the node is not found.

The second optimization improves the successor and min operations when the node following the beginning of the "immutable range" is alive. In this scenario, re-checking the head of the range is unnecessary. Instead, we can simply return the next node.

3 Integration with search trees

For any search tree capable of providing a close value when an exact match is not found, our proposed algorithm can be utilized to implement searches for predecessor and successor elements. To achieve this, during the operation we retrieve a hint from the tree and use it to accelerate traversal in the list.

3.1 Insertion and Deletion Logic

To achieve this, during the insertion of an element, we first retrieve a node from the tree that is close to the target key (a so-called "hint"). We pass this hint to the prepare operation of TreeElement before inserting the element into the tree as a pointer.

When deleting an element, we first remove it from the HintedList and only then remove it from the tree.

In more detail, to insert an element with key K, we first locate a hint node H in the tree (any node with a key close to K). Using this hint H, we attempt to insert a new Node with key K into the HintedList. If the insertion fails because an element with key K already exists in the list, the algorithm returns false and terminates. Otherwise, we proceed to insert the pointer into the tree using the following loop:

We attempt to insert a pointer to the Node into the tree. If successful, we check if the node is still alive. If it is alive, we simply return true; otherwise, we attempt to remove it from the tree and return true regardless of the result. If insertion into the tree fails, it implies that an element with key K already exists in the tree. If our node is still alive then this node in the list is already deleted from the list, but not from the tree, so we can delete this expired hint and restart loop to insert our node. If our node is not alive, we can exit with true.

To delete an element with key K from the tree, we identify a hint node in the tree close to K. Then, we invoke the erase operation of the HintedList using this hint and key K. If this operation returns false, the element does not exist in the list, so we return false. Otherwise, we attempt to remove the element with this key and value from the tree and return true.

3.2 Predecessor and Successor Queries

To retrieve the minimum or maximum elements, we simply obtain the successor of the head or the predecessor of the tail, respectively. To find the predecessor or successor of an element we first obtain a hint from the tree and then invoke the corresponding operation on the Hinted List using this hint.

Listing 5 Pseudocode for insert and erase operations using Hinted List

```
163 bool insert(Key K, Value V) {
164     Node* hint = Tree.GetCloseValue(K);
165     Node* newNode = new Node(K, V);
```

```
166
167     if (!HintedList.insert(newNode)) {
168         retire(newNode);
169         return false;
170     }
171
172     while (true) {
173         successful, AnotherValue = Tree.TryInsert(K, newNode)
174         if (successful) {
175             if (READ(newNode->next_).state_ == Ok) return true;
176             else {
177                 Tree.TryRemoveKeyValue(K, newNode);
178                 return true;
179             }
180         } else {
181             Node* existing = AnotherValue;
182             if (READ(newNode->next_).state_ == Deleted) return true;
183             Tree.TryRemoveKeyValue(K, existing);
184         }
185     }
186 }
187
188 bool erase(Key K) {
189     Node* hint = Tree.GetCloseValue(K);
190     successful, removed_node = HintedList.erase(K, hint);
191     if (!successful) return false;
192
193     Tree.TryRemoveKeyValue(K, existing);
194     return true;
195 }
196
197 Optional<Value> search(Key K) {
198     Node* hint = Tree.GetCloseValue(K);
199     return HintedList.search(K, hint);
200 }
201
202 Optional<Value> find_min() {
203     return HintedList.successor(HintedList.head_);
204 }
205 Optional<Value> find_max() {
206     return HintedList.predecessor(HintedList.tail_);
207 }
208
209 Optional<Value> successor(Key K) {
210     Node* hint = Tree.GetCloseValue(K);
211     return HintedList.successor(K, hint);
212 }
213 Optional<Value> predecessor(Key K) {
214     Node* hint = Tree.GetCloseValue(K);
215     return HintedList.predecessor(K, hint);
216 }
```

4 Correctness Analysis

4.1 Hinted Ordered List Backpointers Correctness

We define the following terms:

- *Active*(N): A node N is *Active* if it is reachable from the head of the list by following next_ pointers.

- *bound*(N): The largest key of active node reachable from node N by following prev_ pointers.

So let's formalize a validity: A Node X is considered *valid* if any of the following conditions hold:

1. X->prev_->next_.ptr_ == $X \wedge$ *Active*(X->prev_)
2. X->prev_->next_->next_.ptr_ == $X \wedge$ *Active*(X->prev_)
3. X->prev_->prev_->next_.ptr_ == $X \wedge$ *Active*(X->prev_->prev_)$\wedge$ $\not$Active$X$->prev_
4. X->prev_->prev_->next_->next_.ptr_ == $X \wedge$*Active*(X->prev_->prev_)$\wedge$ $\not$Active$X$->prev_

This formally defines what previously was shown on the figure.

To ensure that backpointers work well we prove that no operation violates the four following invariants:

1. If A->next_.ptr_ == B and B->next_.ptr_ == C, then upon the successful eviction of B, the following holds: C->prev_ == B and B->prev_ == A.
2. Given a time t_0, and nodes A and B active at t_0 such that A->key_ $\leq$ *bound*(B):

   $$\forall t > t_0, \text{ if } A \text{ is active at } t \implies \textit{bound}(B) \geq A\text{->key_ at } t.$$

   Intuitively: The bound of any active node never decreases below the key of any active node that was greater than (or equal to) it at some point in the past.
3. If barrier(B) returns true at some point t_0 (t_0 being the linearization point of the barrier), then for all active nodes C at t_0 such that C->key_ > B->key_, it holds that *bound*(C) $\geq$ B->key_. *Intuitively:* After a successful barrier on B, all active nodes with keys greater than B have a bound $\geq$ B->key_.
4. $\forall C$, *Active*(C) $\implies$ *Valid*(C). *Intuitively:* All active nodes are always valid.

Any of these invariants can only be violated by a successful CAS operation on the next_ or prev_ pointer of some node. The CAS operations in the algorithm can be categorized into three groups:

I. CAS on the prev_ pointer during a barrier operation.
II. CAS on the next_ pointer during the insertion of a new node.
III. CAS on the next_ pointer during the eviction of a node.

Additionally, the third invariant could potentially be violated upon the return of a barrier operation; however, we will show that this cannot occur. There is also one additional CAS operation on the next_ pointer that switches the state from Ok to Deleted during node deletion. This operation does not violate any invariants because it does not change the value of next_.ptr_, and the invariants are formulated such that the state_ field is not involved. It is important to note that this field is still very important as it provides us with guarantees of Harris's lock-free list which are required for the correctness of current algorithm.

We now analyze each group of CAS operations to show they preserve the invariants.

Group I: CAS on prev_ during barrier

1. The first invariant cannot be violated because this CAS does not modify next_ pointers.
2. Assume for the sake of contradiction that this invariant is violated. We must prove that the barrier does not decrease the bound of any node. It suffices to prove that barrier(B) does not decrease the bound for C := B->next_. If another node's bound were changed

by this operation, it implies the path to the closest Active node passed through the C->prev_ pointer. Thus, if $bound(C)$ does not decrease, the bound of any other node also remains undiminished.
We prove that when the barrier successfully performs a CAS on C->prev_, B is Active. It is important to note that to_check in every call of the barrier is the last alive node before B. And this property persists while to_check is the same. On the last read of to_check we are sure that B is either the next after the node to_check.self_ or B is the next after the next, which is also in a Deleted state. Anyway if this check is successfull B is Active as it is reachable from the node which is in Ok state, which means it is Active too. It is also important that if the check was successful then at this moment $B.next_$ was the same as when we read it before. As in case if B is Ok the check was performed on itself, and if B is Deleted then it is immutable so cannot change. Consider the last node evicted with C in its next_ pointer prior to this specific barrier CAS operation. Let us call this node D. It is most likely B, but it could be another node between B and C.

- If D is B, then prior to the eviction of B, there was another successful barrier on B with C as its next_ pointer. Consequently, C->prev_ was updated there. The current CAS cannot succeed because C->prev_ is no longer __collisionless__.
- If D is not B, then prior to the eviction of D, there was a successful barrier on D with C as its next_ pointer. Similarly, C->prev_ was updated there. Note that D cannot be the previous value of C->prev_ because D was inserted after the last read of C->prev_ in the current barrier operation.

3. This invariant cannot be violated here.
4. this invariant will be proven at the end of this section

Group II: CAS on next_ during insertion

1. No prev_ pointers are changed in this CAS operation, so the invariant holds.
2. No nodes are evicted and no prev_ pointers are changed, so the invariant holds.
3. Cannot be violated here.
4. Assume there exists a node X which is active but not valid after this CAS operation. Before this CAS, at least one of the four validity conditions held. If conditions (1) or (3) held, the invariant is preserved because these states transition to conditions (2) or (4), respectively, following the CAS. We now prove that conditions (2) and (4) were impossible before this CAS among the involved nodes. Consider the transition $A \to B \to C$ becoming $A \to N \to B \to C$. No nodes other than B and C can be invalidated, as the validity conditions for others do not involve the A->next_ pointer. Before insertion, there was a successful barrier on A, implying $bound(B) \geq A$->key_. Therefore, at the moment of insertion, either B.prev_ == A or B.prev_.prev_ == A. This corresponds to case (1) or (3). The same logic applies to C, as there was a successful barrier on B before insertion.

Group III: CAS on next_ during eviction

1. Consider the eviction of B from $A \to B \to C$, resulting in $A \to C$. Since there was a successful barrier on A before eviction, $bound(B) \geq A$->key_. Thus, at the moment of eviction, either B.prev_ == A or B.prev_.prev_ == A. If B.prev_ $\neq A$, it implies another node was inserted after the last read of A->next_, which appeared to be B. Let D be the last node (between A and B) evicted with D.next_ == B prior to the eviction of B. Node D must have been inserted after B was marked as Deleted. However, before D was evicted, the thread initiating the eviction of D would have checked State(B) and

found it to be Deleted. Consequently, it could not proceed with the deletion of D until B was evicted. This yields a contradiction: since B is currently being evicted, no node could exist between A and B after B was marked Deleted if the eviction of B from $A \to B \to C$ succeeds. Therefore, when barrier(A) completed, B->prev_ became A and remained so until the eviction of B. After B was marked as Deleted, no node can be inserted between B and C. Thus, after barrier(B) completed, C->prev_ became B and remained so until the eviction of B.

2. Assume this invariant is violated during the eviction of B from $A \to B \to C$. Since there was a successful barrier on A before eviction, $bound(D) \geq A$. For every node D, the prev_ pointers lead to an active node with a key $\geq A$. Even if this node is B, the invariant holds because B.prev_ $== A$.
3. Cannot be violated here.
4. Assume there exists a node Y that is active but not valid after the CAS operation evicting B from $A \to B \to C$. Before this CAS, Y satisfied one of the four validity conditions. If it satisfied (1) or (2), nothing breaks, as these will hold or transition to (3) or (4) after the CAS (since B->prev_ $== A$). We prove that (3) and (4) were impossible before this CAS (or the CAS would not succeed) for the involved nodes: Y cannot be a node with key $> C$.key_. After barrier(C) returns true, $bound(Y) \geq C$.key_. Thus, either Y->prev_ is active with key $\geq C$.key_, or Y->prev_->prev_ is active with key $\geq C$.key_. In both cases, Y is not invalidated. Also, Y obviously cannot have a key $\leq A$.key_ because its prev_ pointers cannot lead to any nodes with key $\geq A$.key_. Therefore, Y.key_ $\in (A$.key_, C.key_$]$. This implies Y is either B or C. Y cannot be B because B is not active after this CAS. If Y is C, then before this CAS it held either (3) or (4). If it was (4), then C->prev_->prev_ $== A$. Case (3) will hold after the CAS. If it was (3), then C->prev_->prev_ $== B$. Consider the barrier on B called by the thread currently evicting B. After this barrier, C->prev_ became B. For (3) to hold, another barrier on a node D must have changed C->prev_ from B to D. However, this implies D was evicted before B was marked. The value of C->prev_ read by this "expired" CAS operation was B, but was later changed to D by a barrier from another thread. Since C->prev_ changed, the CAS cannot succeed. Thus, C->prev_ must be kept equal to B until the eviction of B is completed, meaning (3) could not hold before this CAS.

Now we prove that the fourth invariant holds after CAS in Group I. If this CAS finished successfully, then the last check in the barrier guaranteed that B was active at that moment. If some thread evicted B after this check, that means that it also performed a successful barrier on B before evicting it. After that previous barrier on B, the backpointer of the next node (C) was pointing to B and could not be changed until the eviction of B, as no other barrier could read this pointer and then perform a check successfully and finish CAS. This is so because at the moment of the last check of a successful barrier on D, it is guaranteed that the node whose prev_ pointer the barrier performs CAS on is D's next (can be proven the same way as previously), and no other node can be inserted between B and C. Thus, after the successful CAS in the barrier on B, C->prev_ $== B$ and B is active. Thus, C is valid as it is B->next_. Thus, no valid node can be violated, because no backpointer of another active node is involved in any validity condition.

Finally, we show that the third invariant cannot be violated upon the return of a barrier operation. Assume it is violated upon the return of barrier(B) called by thread T. Upon the insertion of B, there could be only one active node with a key greater than B.key_ that had a bound $< B$.key_. This node is $C :=$ B->next_.ptr_ at the moment of insertion. Every other node either had a key less than B.key_ or had a bound $\geq B$.key_ due to validity. If a

node has a key $> C$.key_, then its prev_ or prev_.prev_ (the first active one) is either C or B. It cannot be $< B$, otherwise next_->next_ could not reach it back.

We prove that *bound*(C) is also secured after barrier(B) ends. If, before barrier(B) ends, C ceases to be B->next_, then another node D was inserted between B and C. Due to validity, either C->prev_ is alive with key $\geq B$.key_, or C->prev_->prev_ is alive with key $\geq B$.key_. Thus, *bound*(C) $\geq B$.key_, and this is maintained due to Invariant 2. Alternatively, if C remains B->next_, then after barrier(B) ends, C->prev_ becomes B, ensuring *bound*(C) $\geq B$.key_.

Thus, all invariants hold. The backpointers are correct. Since this algorithm utilizes Harris's list and backpointers are used solely as hints, the correctness of the Hinted Ordered List is proven.

All linearization points for insert and erase operations are inherited from Harris's list. The linearization point of the insert operation is the successful CAS that links the new node into the list. The linearization point of the erase operation is the successful CAS that marks the node as logically deleted.

4.2 Correctness of search, successor, and predecessor

To prove the correctness of read-only operations, it suffices to show that they also inherit linearization points from Harris's list. The linearization point for these operations is defined as the last read of the end of the found "immutable range". We guarantee that at this point, no keys logically existed in the list within this range (exclusive) for the following reasons:

If both reads of the first element of the interval yield the same result (the node is alive and points to the same successor), then at the moment of reading the last element, we guarantee that its state corresponds to the read result, and all intermediate nodes (excluding the first) are logically deleted. The first node has not changed since the first read of the second node. If a value had been inserted between the first and second nodes and subsequently removed, the second node must have been removed *before* the start of the deletion of the newly inserted node (or any following removed nodes). This is because the deletion algorithm checks that the successor is not marked for deletion before marking a node for removal. Therefore, since the reading of the last node in the "range" occurs between the first reading of the second node and the last reading of the first node, the state of this range (including the borders) at the moment of the last read is identical to what was observed during traversal.

Thus, we guarantee that decisions made based on the state of nodes in this range are valid and can be considered atomic, with the linearization point being the last read of the end of the range.

4.3 Correctness of optimizations

Now we prove the correctness of the two optimizations presented earlier.

The first optimization, which accelerates node lookup, preserves correctness but has a less obvious linearization point.

In this optimization, we locate an alive node with a key less than or equal to the desired key and then iterate forward from it. Let A be the last alive node encountered during this traversal, and let B be the node where traversal stops. If we reach B and it contains the desired key, the read of this node's state serves as the linearization point; this case is trivial.

The two remaining cases are similar: either B is deleted, or B has a key greater than the desired key. In these cases, we prove that there existed a moment between reading $A->next_$ and reading $B->next_$ when the list did not contain the desired key. To

demonstrate this, we show that if at any moment a living node D is reachable from any (not necessarily living) node C via next pointers, then D remains reachable from C at any subsequent moment until D is deleted. This follows from the fact that no operation except CAS on eviction and CAS on insertion can modify *next_* pointers. Neither of these operations can disrupt the existing path from C to D without first marking D as deleted. Consequently, if a node with the desired key existed in the list during this interval, it would have been reached from A before reaching B.

Therefore, if we do not find a living node with the desired key, it implies that at some moment between reading $A->next_$ and reading $B->next_$, no such node existed in the list. We can thus establish this moment as the linearization point.

The second optimization, which accelerates finding the successor and minimum, modifies the original algorithm only when there are two consecutive living nodes ($A->B$). In this scenario, if we read $A->next_$ followed by $B->next_$, we can be certain that B was alive at the moment of the first read, as a deleted node cannot be resurrected. Thus, this moment serves as a valid linearization point.

5 Evaluation

5.1 Theoretical Analysis

Compared to a more general aggregation function algorithm on search trees, which typically requires a logarithmic number of CAS operations, the proposed algorithm imposes only a constant CAS overhead on all modification operations while preserving the read-only nature of read operations.

In the absence of contention, the insertion operation requires two CAS operations compared to the underlying search tree, whereas deletion requires three. Both algorithms allow for retrieving the minimum and maximum elements in constant time. Searches for predecessor and successor elements are performed in logarithmic time on average in both algorithms, as they are strictly bound by the performance of the underlying search tree.

It is also worth noting that the doubly linked list algorithm developed as part of this work demonstrates an advantage over the current state-of-the-art [8]. Specifically, while insertion similarly requires two CAS operations, our approach reduces the cost of deletion to only three CAS operations, compared to four in the existing solution. Moreover, our implementation allows for avoiding the use of double-word CAS (DWCAS) primitives if necessary.

5.2 Empirical Analysis

In this section, an empirical analysis of the proposed algorithm is presented. To empirically evaluate the performance, we implemented the algorithm on top of the setbench [2] project. We implemented an augmented tree [4] based on Ellen et al.'s [3] non-blocking binary search tree and implemented the algorithm described in this paper based on the same tree as well. Then we ran a series of measurements using microbench from setbench [2]. The measurements were performed with different sizes of the tree, different numbers of threads, and different percentages of update operations. Each measurement was performed five times and averaged. The fork of setbench [6] containing our implementations is publicly available.

Platform: The measurements were performed on a machine with four Intel(R) Xeon(R) CPU E7-4830 v3 @ 2.10GHz processors, each with 12 cores and 24 logical threads. The machine has 504G of RAM, 1.5M L1 instruction cache and 1.5M L1 data cache, each distributed among 48 cores, 12M of L2 cache distributed among 48 cores, and 120M L3 cache

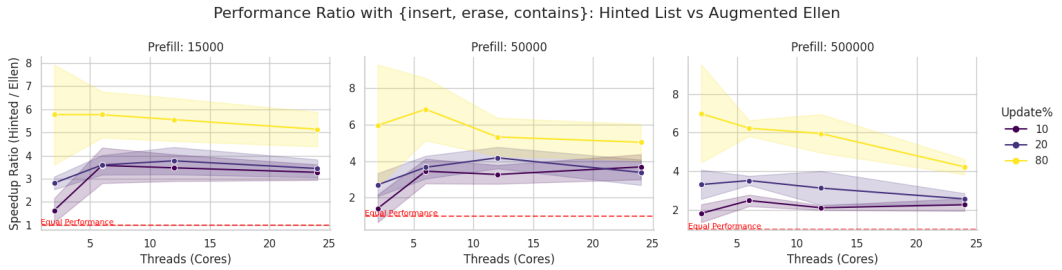
distributed among 4 sockets. To avoid the effects of hyper-threading, we pinned the threads to physical cores, avoiding scheduling anything on the same physical core. All the measurements with 12 threads or less were performed on a single processor, while measurements with more than 12 threads utilized multiple processors.

Software: The code was compiled using g++ (Ubuntu 13.3.0-6ubuntu2 24.04) 13.3.0 with the C++17 standard, -O2 optimization level, and mcx16 extension enabled for double-word CAS operations.

Methodology: Each measurement was performed for 20 seconds. Before each experiment, the tree was prefilled with half of the key range randomly. The key ranges we chose are (0, 30,000], (0, 100,000], and (0, 1,000,000].

The operations were randomly chosen from the sets insert, delete, contains, insert, delete, successor, or insert, delete, min. The percentage of update operations varied from 5 to 40 percent each, which results in 10-80% of read operations. The keys for the operations were chosen randomly from the key range.

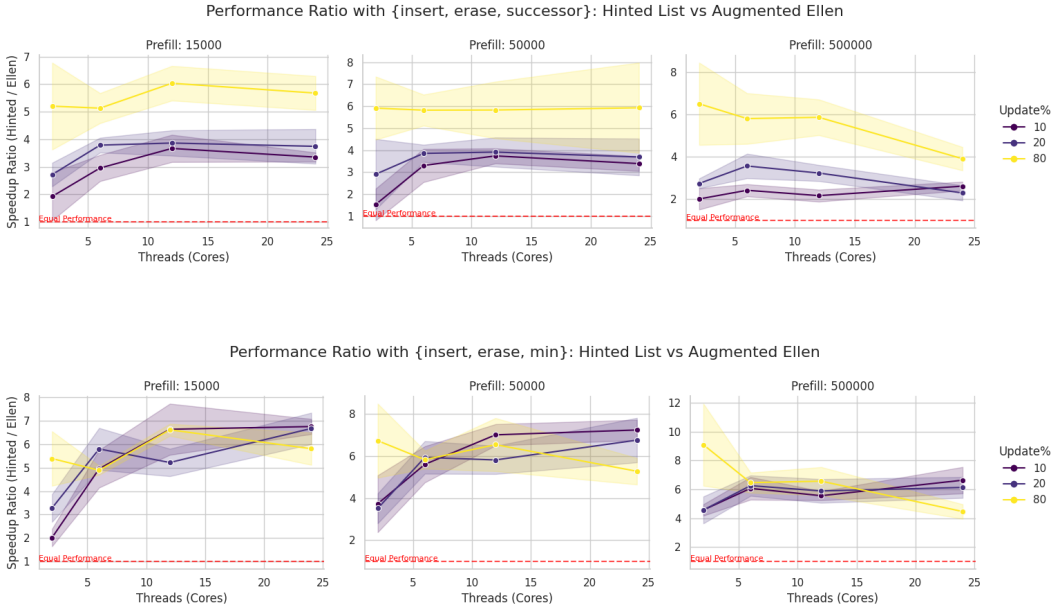
After the measurements were performed, we calculated the ratio of the total throughput of the new algorithm to that of the augmented tree for each measurement. We took the average of these ratios for each configuration of the parameters (number of threads, size of the tree, percentage of insert/delete operations). Standard deviation was also calculated to show the results.



The results show that the new algorithm outperforms the augmented tree on most workloads; however, the augmented tree performs better on read-heavy workloads with a small number of threads. These results are expected as read operations on the tree are performed on *version* objects, which are not synchronized, so all operations, except reading the *version* object of the root, are performed without any synchronization. However, write operations are significantly more expensive as multiple versions must be modified for each write operation. With an increase in the number of threads, this effect becomes less significant because threads start to "help" each other with the propagation of the versions. The most unexpected result was that the new algorithm performs surprisingly well on a small number of threads with write-heavy workloads compared to the augmented tree. We assume that this happens because in this scenario each write operation in the augmented tree needs to propagate the changes to the root individually.

The results of the measurements for the successor operation are quite similar to those of the contains operation. However, here we can see that it is slower compared to the augmented tree than the contains operation. This results in a less significant advantage, especially on read-heavy workloads.

The results of the measurements with the min operation show that the % of min operations does not significantly affect the throughput of the new algorithm compared to the augmented tree. This is expected because both algorithms perform this operation much faster than



any other operation, so with a fixed part of the min operations, the throughput is mostly determined by the other operations.

6 Declaration of AI use

LLM tools were used in redacting and scripting (e.g. script to load-balance measurements or parse their outputs) purposes only. No scientific content was generated using AI.

References

1 Trevor Brown. *Techniques for Constructing Efficient Lock-free Data Structures*. PhD thesis, University of Toronto, Toronto, Canada, 2017. arXiv preprint arXiv:1712.05406. URL: https://arxiv.org/abs/1712.05406.

2 Trevor Brown. Setbench: A benchmark suite for concurrent sets and maps. https://gitlab.com/trbrown123/setbench, 2020. Accessed: 2026-01-09.

3 Faith Ellen, Panagiota Fatourou, Eric Ruppert, and Franck van Breugel. Non-blocking binary search trees. In *Proceedings of the 29th ACM SIGACT-SIGOPS Symposium on Principles of Distributed Computing*, PODC '10, page 131–140, New York, NY, USA, 2010. Association for Computing Machinery. doi:10.1145/1835698.1835736.

4 Panagiota Fatourou and Eric Ruppert. Lock-free augmented trees (abstract). In *Proceedings of the 3rd Highlights of Parallel Computing Workshop*, HOPC '25, page 1–3, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3746238.3746251.

5 Timothy L. Harris. A pragmatic implementation of non-blocking linked-lists. In *Proceedings of the 15th International Conference on Distributed Computing*, DISC '01, page 300–314, Berlin, Heidelberg, 2001. Springer-Verlag.

6 Vladimir Melnikov. Setbench fork for benchmarks. https://github.com/Val-de-mar/setbench, 2025. Accessed: 2026-01-09.

7 Gal Sela and Erez Petrank. Concurrent size. *Proc. ACM Program. Lang.*, 6(OOPSLA2), October 2022. doi:10.1145/3563300.

8 Håkan Sundell and Philippas Tsigas. Lock-free and practical doubly linked list-based deques using single-word compare-and-swap. In *Proceedings of the 8th International Conference on Principles of Distributed Systems*, OPODIS'04, page 240–255, Berlin, Heidelberg, 2004. Springer-Verlag. doi:10.1007/11516798_18.